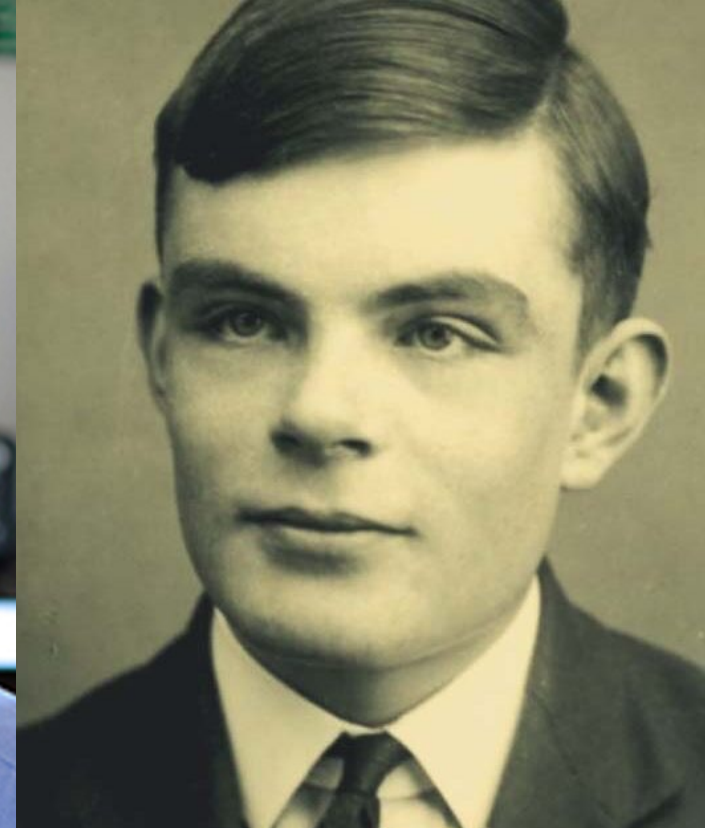


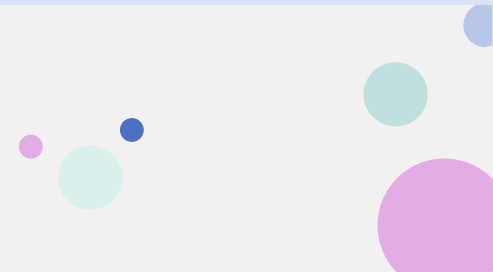


Lecture 3

Tools for Defining Computational Security

CS409: Introduction to Cryptography

Instructor: Sruthi Sekar



Recap/Attendance

- How to define perfect security?
- Building a perfectly secure encryption –Vernam Cipher
- Attacks on Vernam Cipher
- Limitations of Perfect Secure Encryption:
 1. Key must be as long as the message.
 2. Key cannot be re-used.

Relaxations of the Security Model

Impossible to break $\xrightarrow{\text{relax}}$ Infeasible to break with high probability

I We will only consider “**efficient**” **adversaries** instead of all-powerful unbounded time ones

How do we model the adversary? $\rightarrow$ Basics of Complexity Theory

II Adversaries can win (security fails) with some “**small probability**” (be imperfect)

How do we formalize this failure probability $\rightarrow$ Negligible functions

Why do you need both relaxations?

~~I We will only consider “efficient” adversaries instead of all-powerful unbounded time ones~~

II Adversaries can win (security fails) with some “small probability” (be imperfect)

Candidate Definition A

An encryption scheme $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ is secure if, for every **Eve**, it holds that:

$$\Pr_{\substack{m_0, m_1 \leftarrow \text{Eve} \\ k \leftarrow \text{Gen} \\ b \in_R \{0,1\}, c_b = \text{Enc}_k(m_b)}} [\text{Eve}(c_b) = b] = \frac{1}{2} + \varepsilon$$

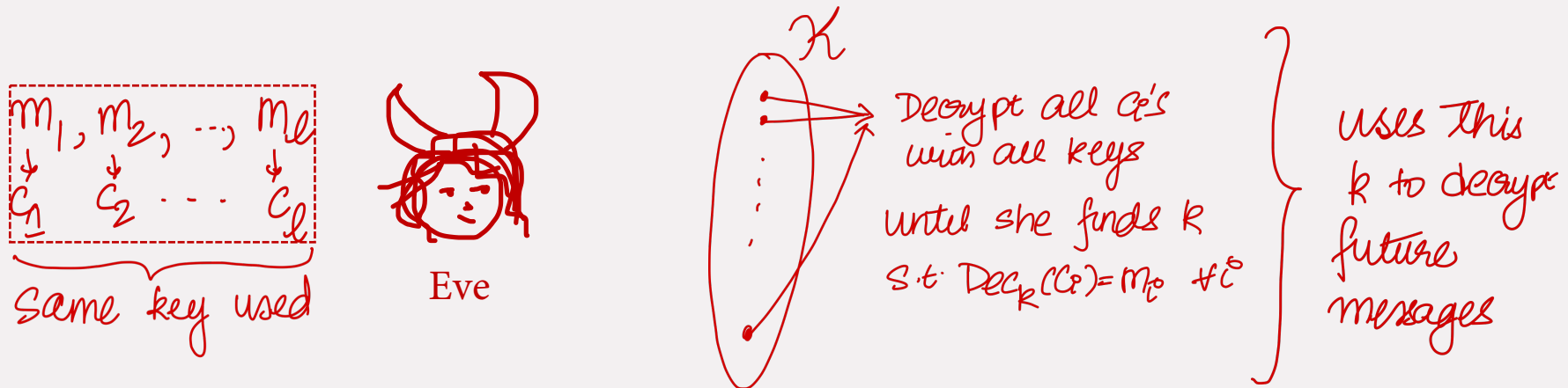
HW 1: Show that for Candidate Definition A, $|\mathcal{K}| \geq (1 - \varepsilon) \cdot |\mathcal{M}|$!

Unbounded Eve is too powerful, even with some error!

Why do you need both relaxations?

- I ~~We will only consider “efficient” adversaries instead of all-powerful unbounded time ones~~
- II Adversaries can win (security fails) with some “small probability” (be imperfect)

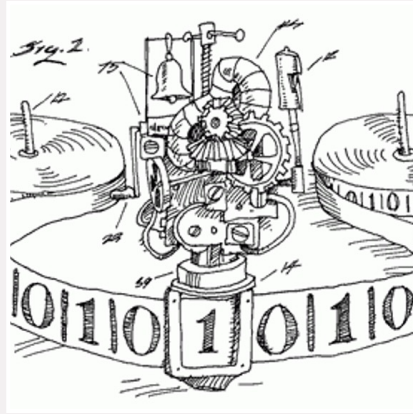
BRUTE FORCE ATTACK (recall we want $|\mathcal{K}| < |\mathcal{M}|$, and to re-use keys)



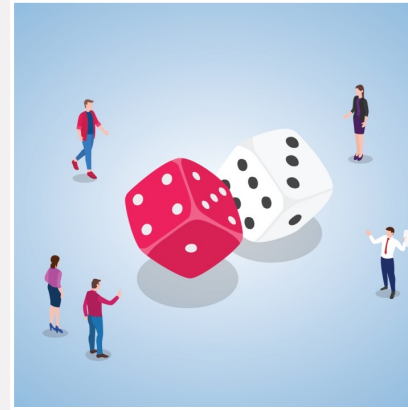
KNOWN-PLAINTEXT ATTACK

Unbounded Eve is too powerful, even with some error!

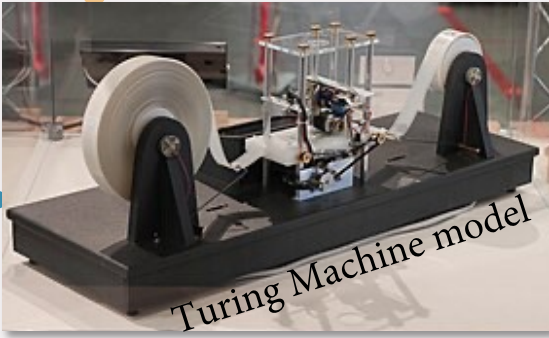
Today's Class



Modeling Computation
(Basics of Complexity Theory)

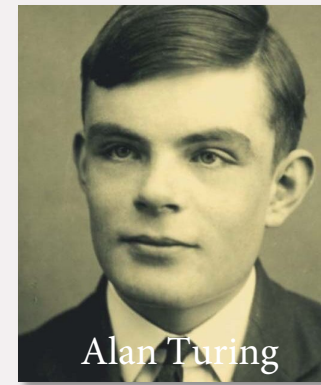


Formalize “low probability success”
(Negligible Functions)

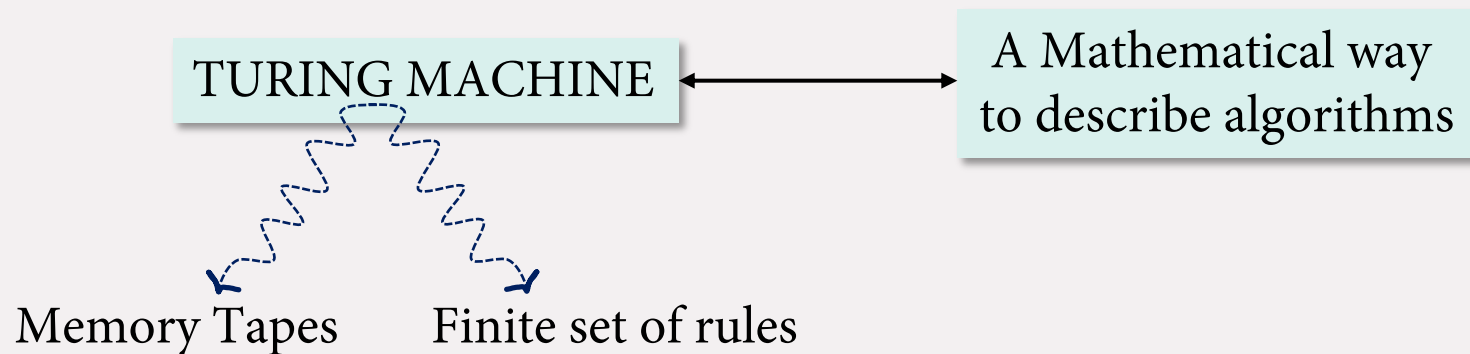


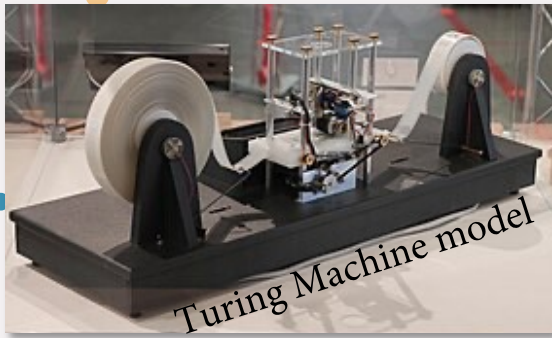
Turing Machines

[1936]



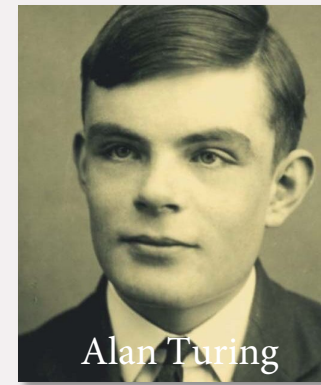
- An algorithm is a set of instructions or rules.
- To understand how an algorithm performs, we need a model of computation. (Turing machine is one such natural model)
- Turing called it an “automatic machine”. His doctoral advisor, [Alonzo Church](#), coined the term Turing Machine!





Turing Machines

[1936]



Definition 1 [Computational Complexity, Arora and Barak](#)

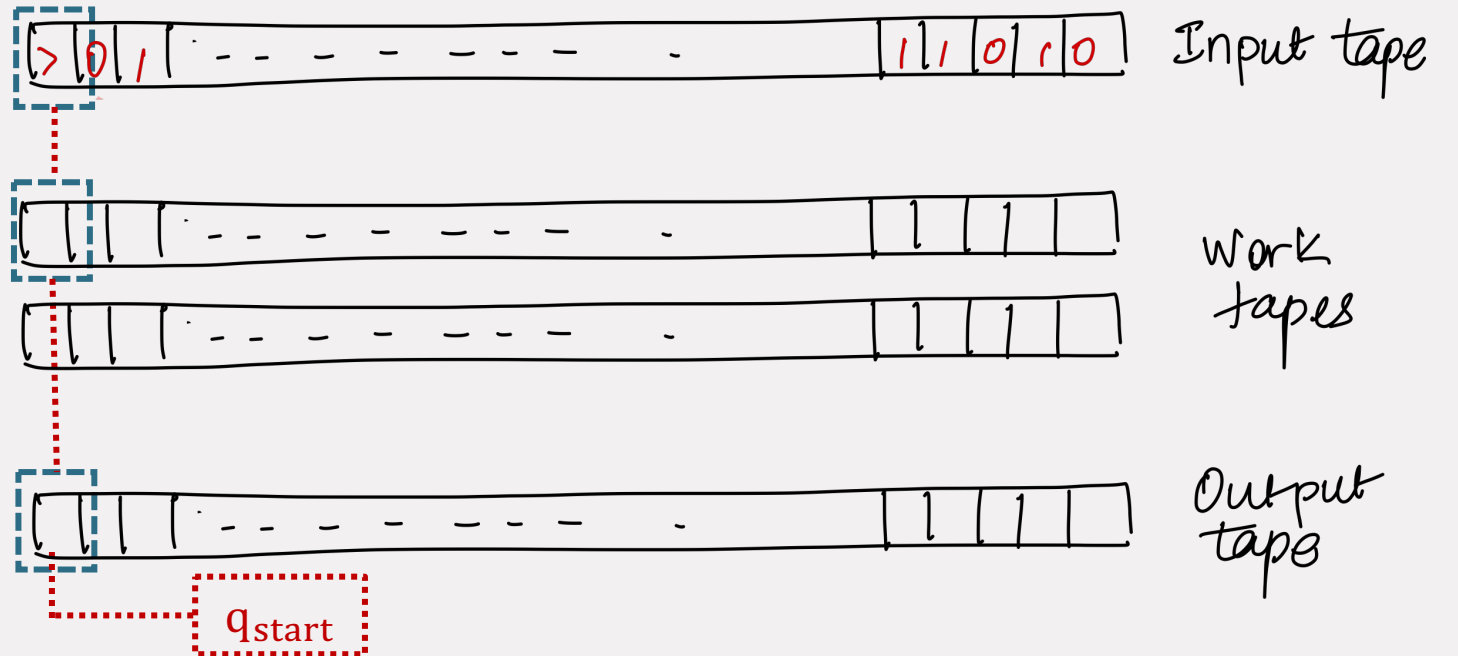
A k -tape Turing Machine M is described by a tuple (Γ, Q, δ) such that:

- M has k memory tapes (input/work/output tapes) with *heads*.
 - Γ is a finite set of alphabets (every memory cell has an element from Γ)
 - Q is a finite set of states (special states: q_{start} , q_{halt}).
 - δ is a function from $Q \times \Gamma^k$ to $Q \times \Gamma^k \times \{L, S, R\}^k$
-  (**transition function**: captures the dynamics of M)
-  has a blank symbol

Turing Machines: Computation

START CONFIGURATION

- Only input tape has input strings.
- All other tapes have blank symbols.
- All the head positions at the start of the tape.
- Machine is in the start state q_{start}



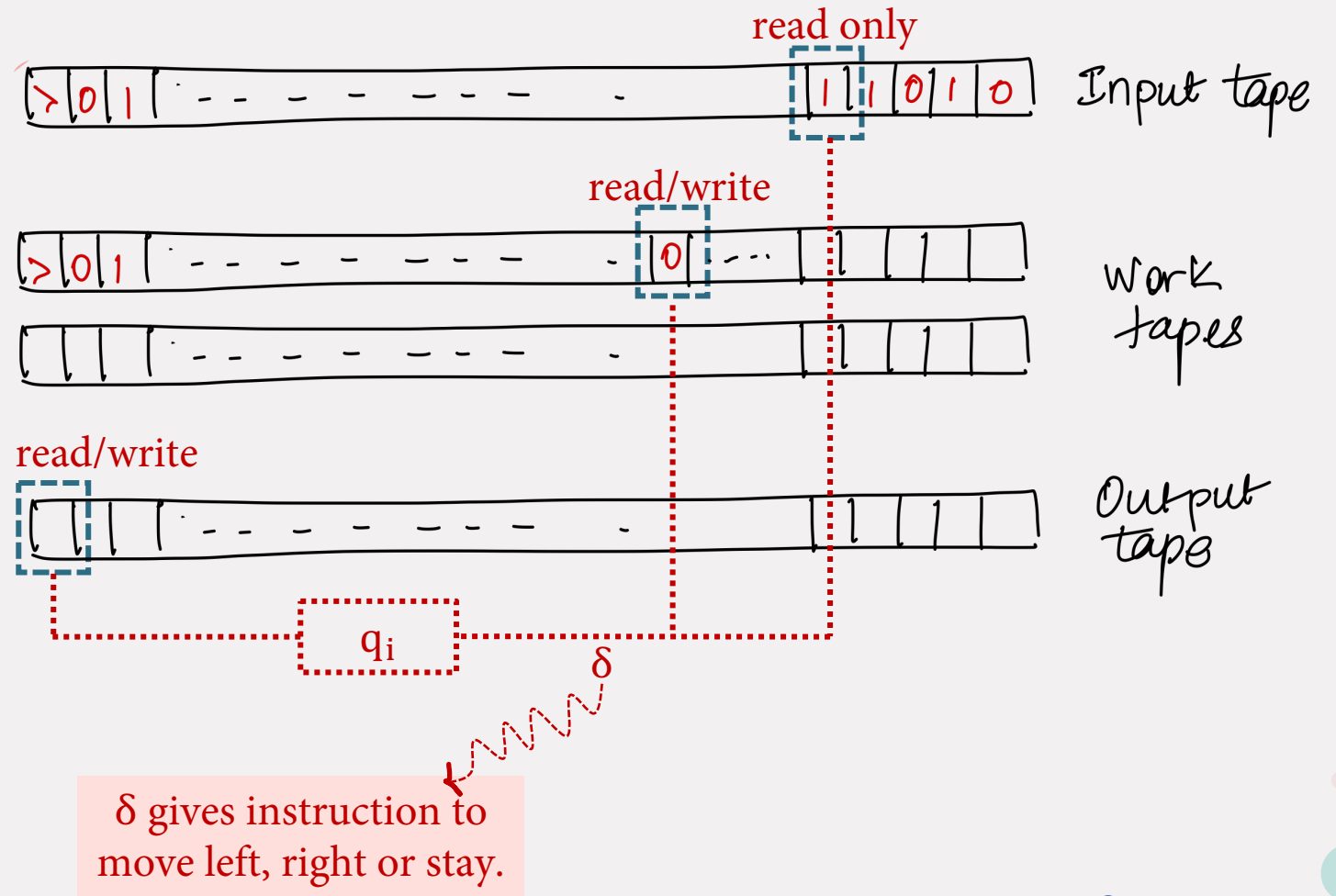
Turing Machines: Computation

START CONFIGURATION

- Only input tape has input strings.
- All other tapes have blank symbols.
- All the head positions at the start of the tape.
- Machine is in the start state q_{start}

COMPUTATION

- A step of computation is performed by applying δ



Turing Machines: Computation

START CONFIGURATION

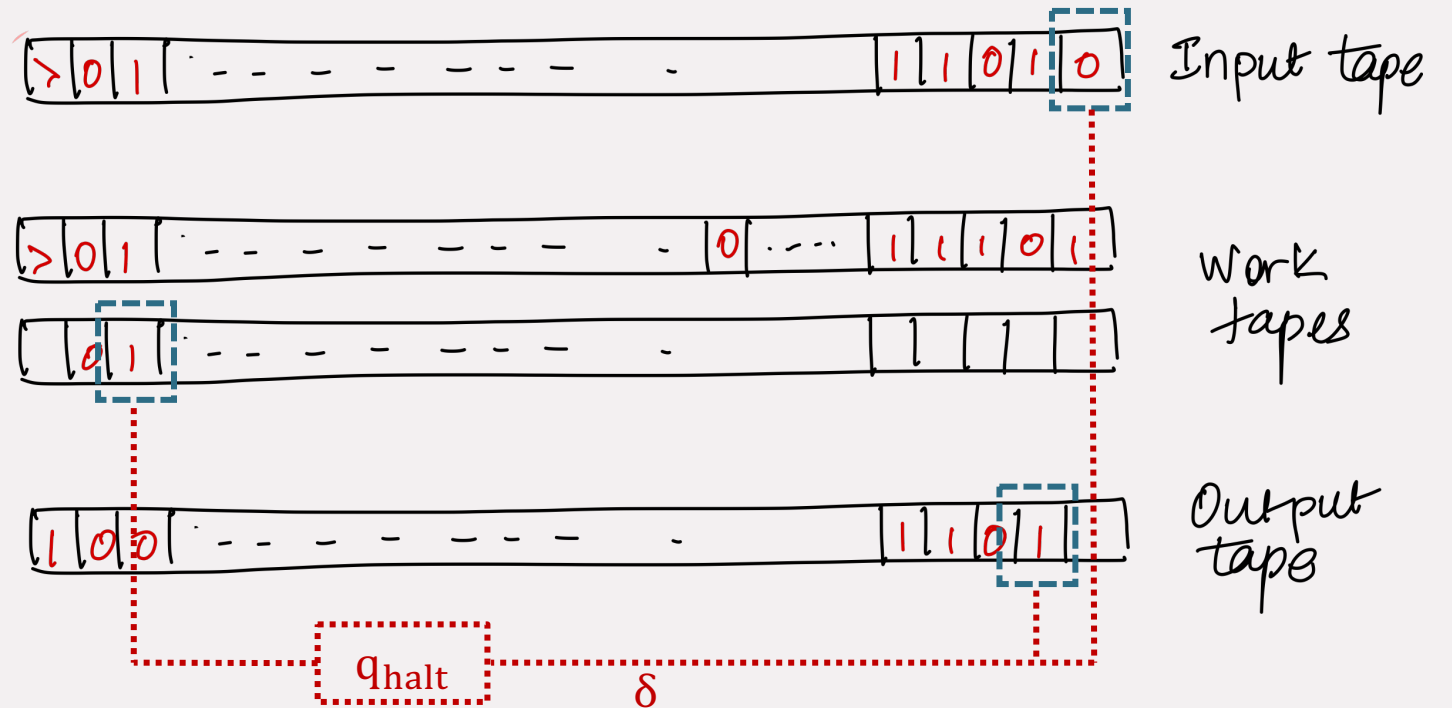
- Only input tape has input strings.
- All other tapes have blank symbols.
- All the head positions at the start of the tape.
- Machine is in the start state q_{start}

COMPUTATION

- A step of computation is performed by applying δ

HALTING

- Once the machine enters q_{halt} , it stops computation.



Turing Machines: Running Times

Definition 2  [Computational Complexity, Arora and Barak](#)

Let $f: \{0,1\}^* \rightarrow \{0,1\}^*$, $T: \mathbb{N} \rightarrow \mathbb{N}$, and M be a Turing machine.

We say M **computes** f if on every input $x \in \{0,1\}^*$, M halts with $f(x)$ on its output tape, beginning from the start configuration with x on its input tape.

M **computes** f in time $T(|x|)$, if for all $x \in \{0,1\}^*$, M halts within $T(|x|)$ steps of computation and outputs $f(x)$.

We will only deal with computational problems that can be computed** using Turing machines

** there are uncomputable problems for which the Turing machine doesn't halt!

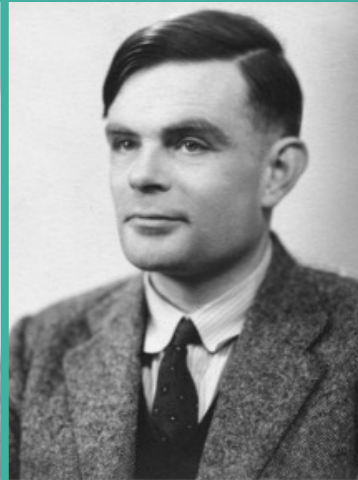


Why Turing Machines?

Several other computation models can be simulated by TMs.

Every physically realizable computation device –whether it's based on silicon, DNA, neurons , or some other alien technology – can be simulated (efficiently) by a Turing machine.

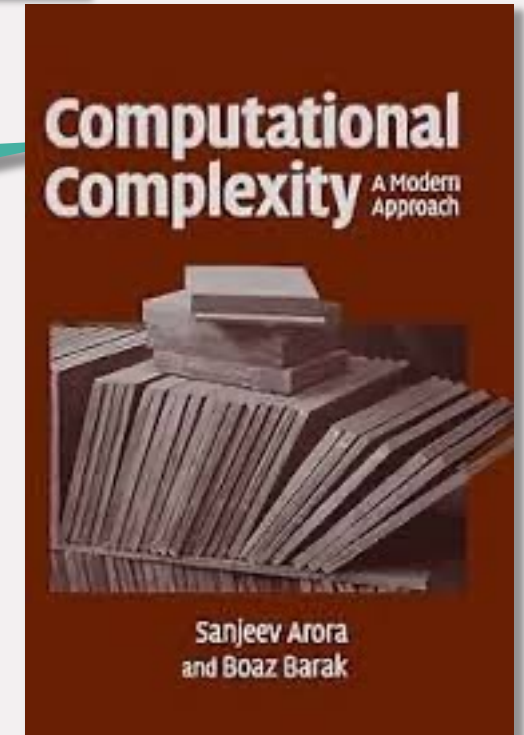
– Church-Turing Thesis



Alan Turing



Alonzo Church



Possible exception: quantum machines!

A photograph of two men in a room with large windows. One man, Wes Clark, is standing and gesturing with his hand while speaking. The other man is sitting on a wooden chair. In the foreground, a wooden table holds a vintage computer system, including a monitor, keyboard, and system unit. The text 'WES CLARK' is overlaid in large white letters, with 'CO-CREATOR OF THE FIRST PERSONAL COMPUTER' in smaller white letters below it.

WES CLARK
CO-CREATOR OF THE
FIRST PERSONAL COMPUTER

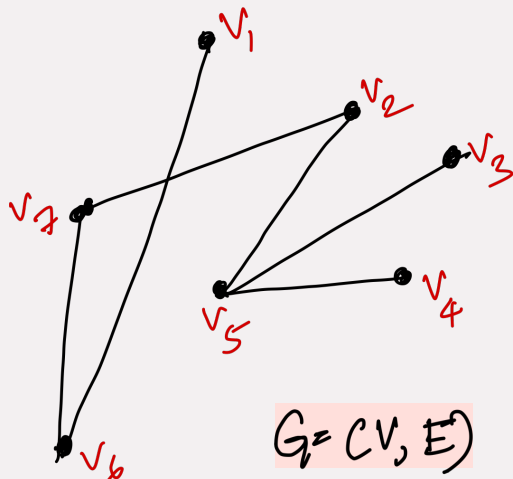
Complexity Classes

Complexity classes contain decision problems or languages.
(think of it as either functions $D: \{0,1\}^* \rightarrow \{0,1\}$ or as subsets $L \subseteq \{0,1\}^*$)

Definition 3: Complexity Class P

Class **P** is defined as a set of languages **L** such that there exists a Turing machine **M** that **runs in polynomial time** (halts in $p(|x|)$ steps for polynomial p), such that $x \in L \Leftrightarrow M(x) = 1$

Example



$L = \{G = (V, E), v_1 \in V, v_2 \in V: \exists \text{ path from } v_1 \text{ to } v_2 \text{ in } G\}$

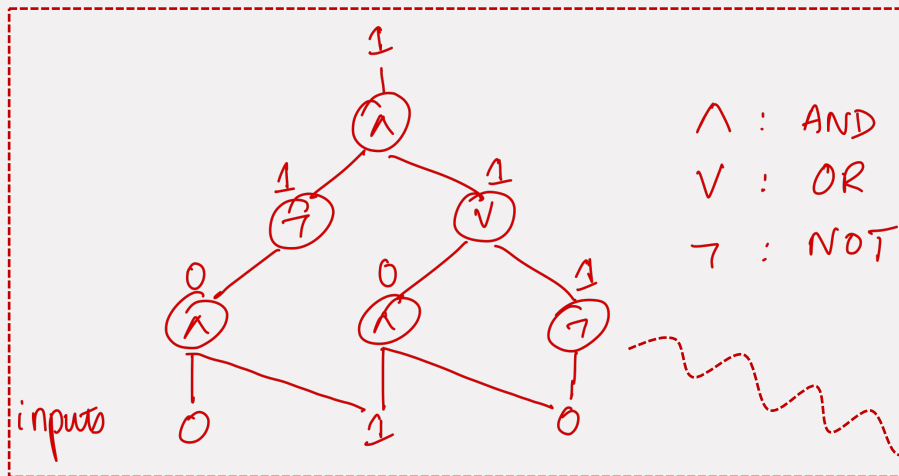
Path finding runs in time $O(|V| + |E|)$,
which is linear in size of the input. $\Rightarrow L \in P$

Complexity Classes

Definition 4: Complexity Class NP

Class **NP** is defined as a set of languages **L** such that $\exists$ Turing machine **M** and a polynomial p such that **M** halts in $p(|x|)$ steps for input x , and $x \in L \iff \exists y \in \{0,1\}^{p(|x|)}$ such that $M(x, y) = 1$

Example



Example of a boolean circuit

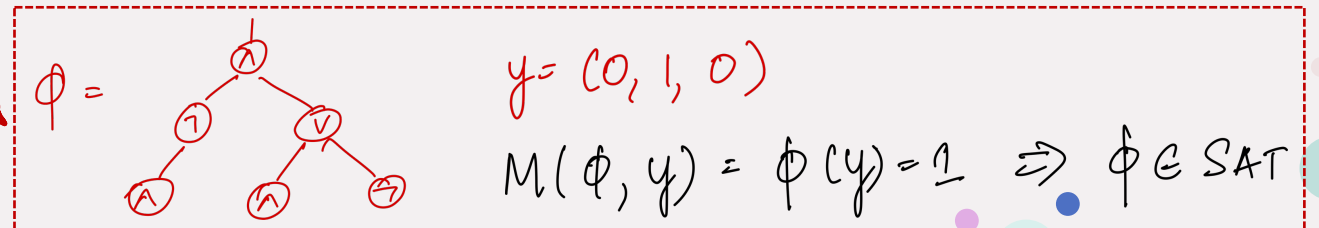
$$L = \text{SAT} = \{\phi : \exists y_1, y_2, \dots, y_n \in \{0,1\}, \phi(y_1, y_2, \dots, y_n) = 1\}$$

Satisfiability

Boolean formula on n variables

$$M(\phi, y = (y_1, y_2, \dots, y_n)) = \phi(y) \implies \text{SAT} \in \text{NP}$$

evaluates and outputs



Complexity Classes

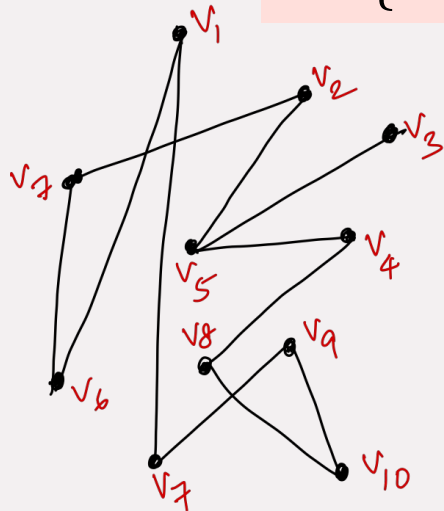
Definition 5: Complexity Class BPP

Class **BPP** is defined as a set of languages **L** such that there exists a **randomized** Turing machine **M** and a polynomial p such that **M** halts in $p(|x|)$ steps for input x ,
 $x \in L \Rightarrow \Pr[M(x; r) = 1] \geq 2/3$ and $x \notin L \Rightarrow \Pr[M(x; r) = 1] \leq 1/3$, for $|r| \leq p(|x|)$

****Chernoff Bounds:** Can be amplified to $1 - 1/2^n$ and $1/2^n$, resp., where $|x| = n$

Example

$L = \{G = (V, E), v_1 \in V, v_2 \in V: \exists \text{ path from } v_1 \text{ to } v_2 \text{ in } G\}$



M(G, v₁, v₂; r):

- Start at v_1
- At each step pick a random edge adjacent to the current vertex and check if v_2 is step.
- Output 1 if v_2 is found, 0 otherwise.

HW 2 (extra): Show a strategy to improve the errors using Chernoff bounds.

HW 3: Is $L \in \mathbf{BPP}$?
Prove it.

Other Computation Models

RAM Turing Machine

This is a Turing Machine with a fixed sized tape, with the additional ability to move the pointer to any position of the work tape in 1 time step.

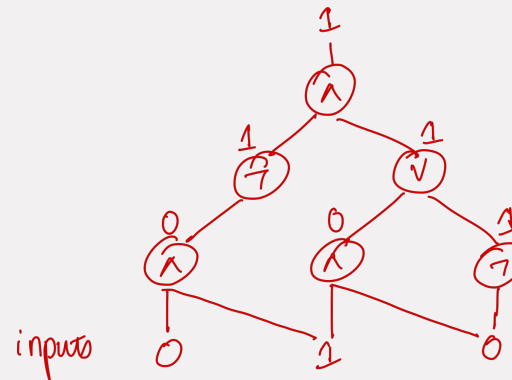
Circuits * Circuits can be modeled as special Turing Machines that take some “advice”

Circuits are represented using wires (input, output, and intermediate) and gates.
For circuits, we measure the complexity by its **size** (number of gates) and **depth** (length of longest path from input to output wire).

Example

A Boolean circuit

$C: \{0,1\}^n \rightarrow \{0,1\}^m$ consists of three gates: AND, OR and NOT.



$\wedge$: AND
 $\vee$: OR
 $\neg$: NOT

Size: 6, Depth: 3

Computation model for Eve

Probabilistic Polynomial Time (PPT) Eve



Eve **runs in polynomial time** ($p(|\text{input}|)$ for some polynomial p) and is allowed to **use randomness**.
($\in \mathbf{BPP}$ if Eve is only outputting 0/1)

Stronger Eve: Polynomial-sized family of circuits



A **polynomial-sized family of circuits** is an infinite sequence of Boolean circuits $C_1, C_2, \dots$ such that for every $n \in \mathbb{N}$, C_n with inputs of size n , and size $p(n)$ (p is fixed polynomial for whole family)

Why do you need both relaxations?

I We will only consider “**efficient**” **adversaries** instead of all-powerful unbounded time ones

~~II Adversaries can win (security fails) with some “**small probability**” (be imperfect)~~

Candidate Definition B

An encryption scheme $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ is **computationally indistinguishable** if for every **PPT Eve**, it holds that:

$$\Pr_{\substack{m_0, m_1 \leftarrow \text{Eve} \\ k \leftarrow \text{Gen} \\ b \in_R \{0,1\}, c_b = \text{Enc}_k(m_b)}} [\text{Eve}(c_b) = b] = 1/2$$



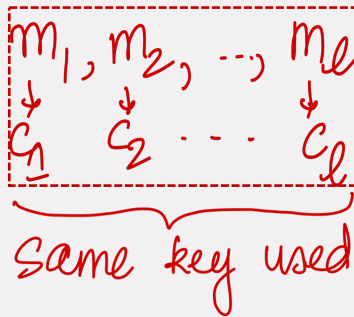
HW 4: Show that Shannon’s impossibility still extends to Candidate Definition B above!

Perfect Break from “efficient” Eve is too much to ask for!

Why do you need both relaxations?

- I We will only consider “**efficient**” **adversaries** instead of all-powerful unbounded time ones
- ~~II Adversaries can win (security fails) with some “**small probability**” (be imperfect)~~

RANDOM GUESS ATTACK (recall we want $|\mathcal{K}| < |\mathcal{M}|$, and to re-use keys)



Guess a random $k \in \mathcal{K}$
and check if $\text{Dec}_k(c_i) = m_i \forall i$
If yes, use this k to decrypt
subsequent messages

$$\Pr[\text{success}] = \frac{1}{|\mathcal{K}|}$$

*Small error win is
always possible

KNOWN-PLAINTEXT ATTACK

Even an “efficient” Eve can always break with small probability

Towards Computational Security

An encryption scheme $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ is **computationally indistinguishable** if for every **PPT Eve**, it holds that:

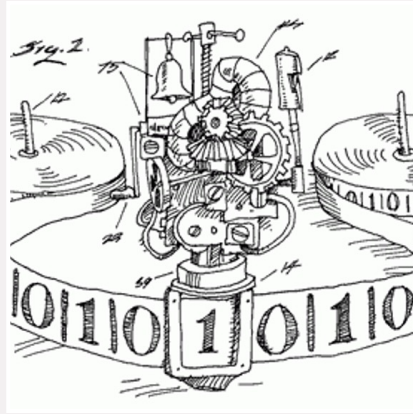
$$\Pr_{\substack{m_0, m_1 \leftarrow \text{Eve} \\ k \leftarrow \text{Gen} \\ b \in_R \{0,1\}, c_b = \text{Enc}_k(m_b)}} [\text{Eve}(c_b) = b] = \frac{1}{2} + \varepsilon$$



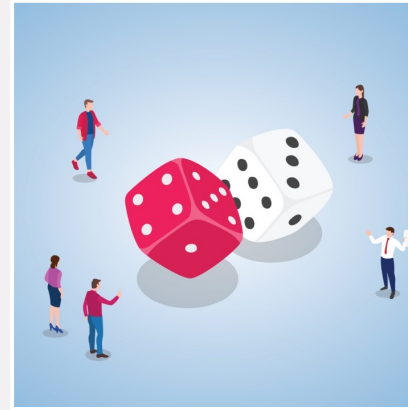
Eve

What ε though? How do we define it?

Today's Class



Modeling Computation
(Basics of Complexity Theory)



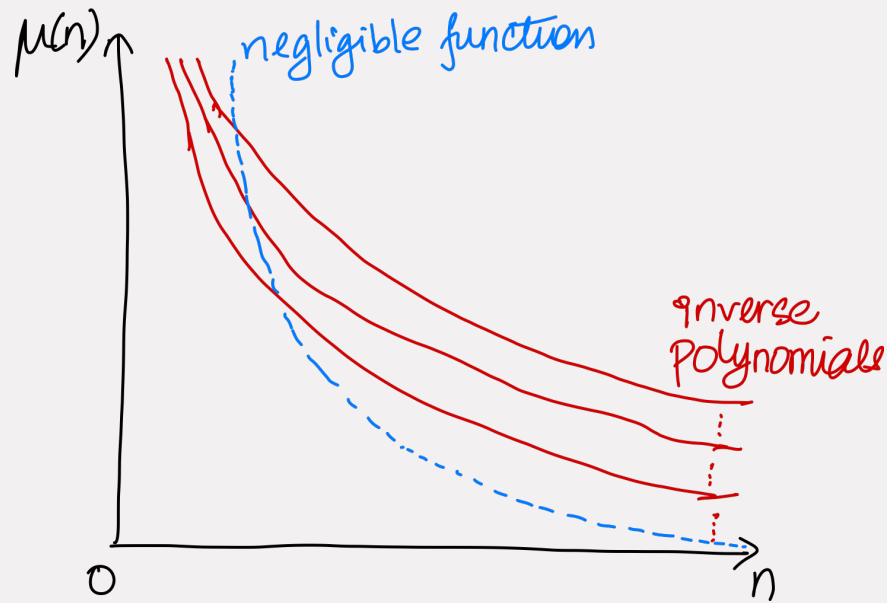
Formalize “low probability success”
(Negligible Functions)

Negligible Functions

Definition 6

A function $\mu: \mathbb{N} \rightarrow \mathbb{R}$ is **negligible** if, for every polynomial function p , there exists an n_0 such that
For all $n > n_0$: $\mu(n) < 1/p(n)$

* Functions eventually smaller than every inverse polynomial.



*Events that occur with negligible probability look to poly-time algorithms like they never occur!

Negligible Functions: Examples

Definition 6

A function $\mu: \mathbb{N} \rightarrow \mathbb{R}$ is **negligible** if, for every polynomial function p , there exists an n_0 such that
For all $n > n_0$: $\mu(n) < 1/p(n)$

✗ $\mu_1(n) = \frac{1}{134572 \cdot n^{134572}}$

✓ $\mu_2(n) = \frac{1}{2^n}$

✗ $\mu_3(n) = \begin{cases} 1/2^n, & \text{for odd } n \\ 1/(134572 \cdot n^{134572}), & \text{for even } n \end{cases}$



Negligible or not

✓ $\mu_4(n) = n^{-\log n}$

✓ $\mu_5(n) = p(n)/2^n$, for very large polynomial $p(n)$

✓ $\mu_6(n) = n^{\log n}/2^n$

Security Parameter

Definition 6

A function $\mu: \mathbb{N} \rightarrow \mathbb{R}$ is **negligible** if, for every polynomial function p , there exists an n_0 such that
For all $n > n_0$: $\mu(n) < 1/p(n)$

Security Parameter is n (sometimes denoted by λ)

- Runtimes of algorithms and success probabilities are measured as a function of n .
- WANT: Honest parties (Alice and Bob) run in time (fixed) polynomial in n .
- ALLOW: Eve can run in time (arbitrary) polynomial in n .
- REQUIRE: Eve to have a success probability negligible in n .

What is the right n ?

Say some designer claims that an **Eve running in n^3 -minutes** can break his scheme with probability $2^{40}2^{-n}$

What n do you choose while implementing?

$n \leq 40$?

Eve working for 40^3 mins (6 weeks) can break with probability 1



$n = 50$?

Eve working for 50^3 mins (3 months) can break with probability $1/1000$



*may be unacceptable

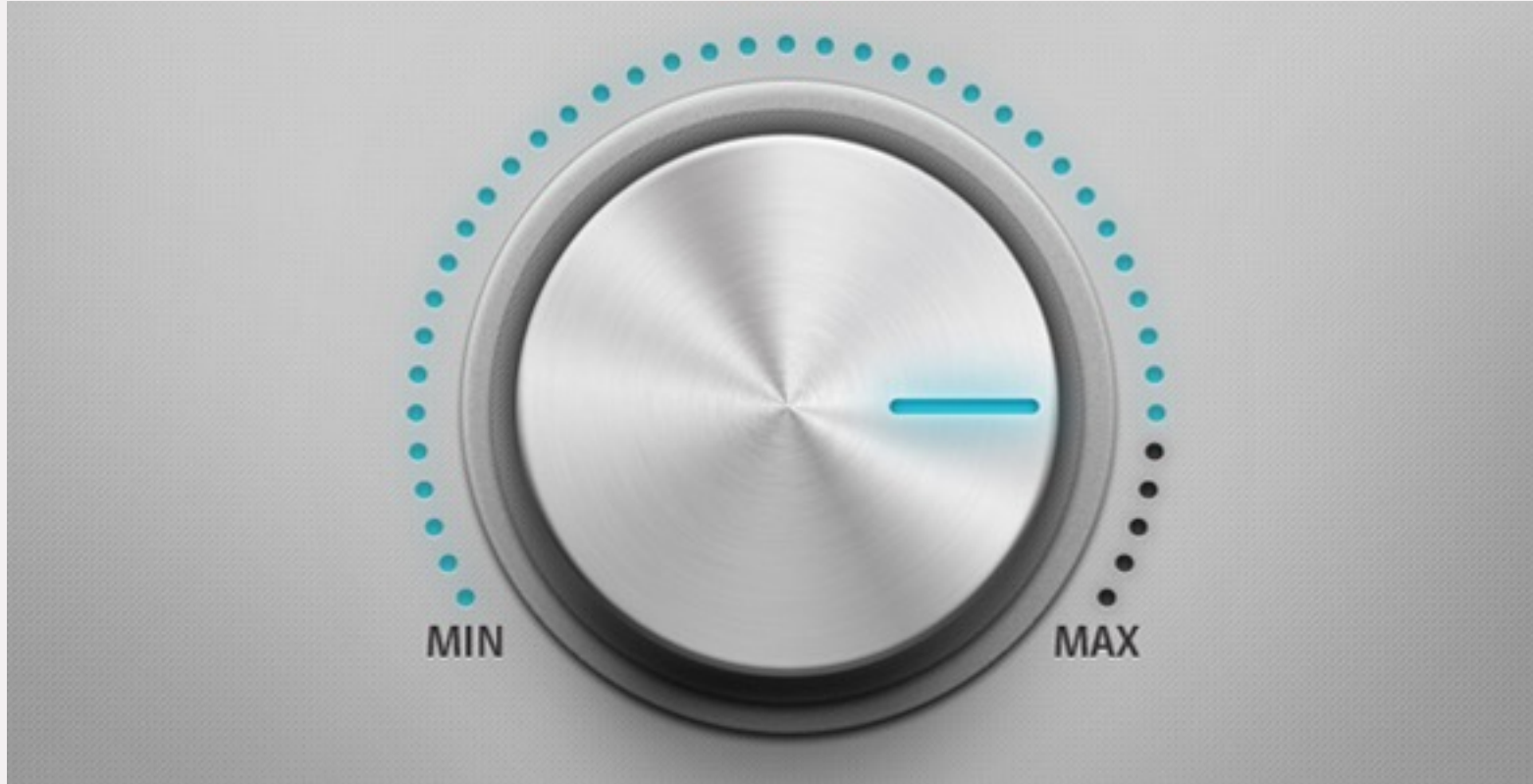
$n = 500$?

Eve working for 200 years can break with probability 2^{-460}



If the probability that an attacker can successfully recover an encrypted message in 1 year is 2^{-60} , then it is much more likely that the sender and receiver will both be hit by lightning in that same time.

What is the right **n**?

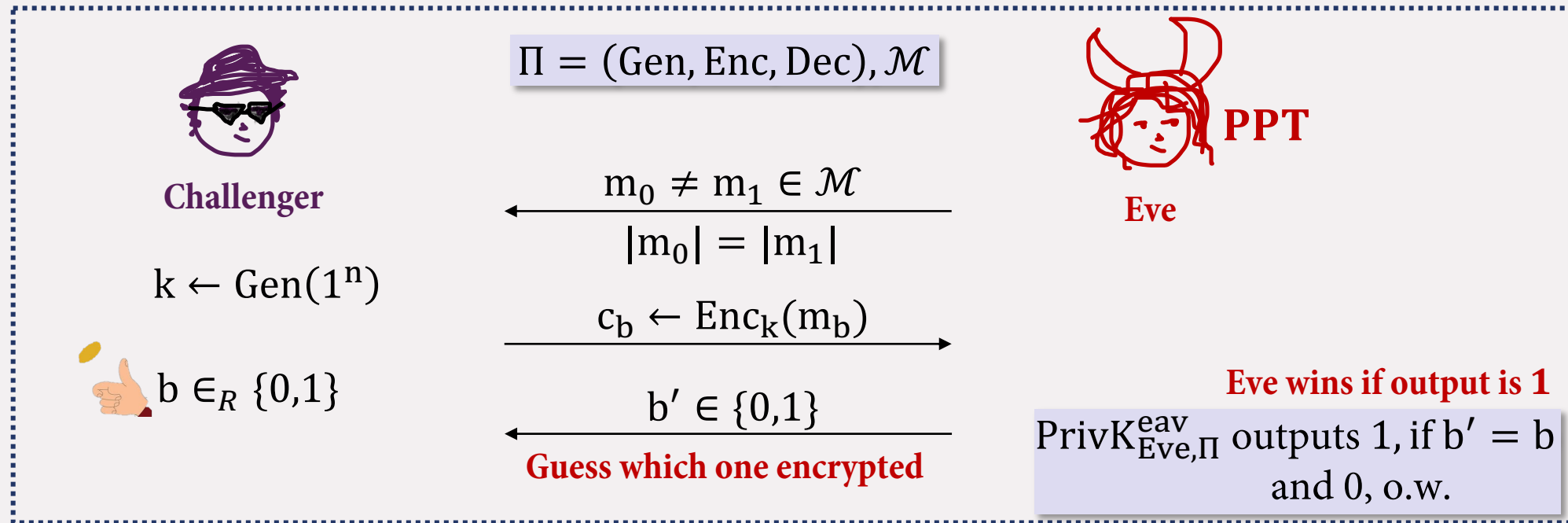


min —————→ max

Adversary's job harder

User Algorithms are slower

Computational Indistinguishability



Definition 7

An encryption scheme $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ is **computationally indistinguishable against an eavesdropper** if, for all **PPT** adversary **Eve**,

there exists a negligible function **negl** such that, $\forall n$

$$\Pr[\text{PrivK}_{\text{Eve}, \Pi}^{\text{eav}}(n) = 1] \leq 1/2 + \text{negl}(n)$$

Next Class..

- Pseudorandom Generators (PRGs)
- Build a computationally secure scheme
- Security Reductions
- Building and applying PRGs

Please register on this link by Thursday night for the lab exercise 1 (will post on Moodle)

<https://cs409.ctfd.io>

IT USED TO BE THAT IF A CLIENT WANTED SOMETHING DONE IN A WEEK, IT WAS CONSIDERED A RUSH JOB, AND HE'D BE LUCKY TO GET IT.



NOW, WITH MODEMS, FAXES, AND CAR PHONES, EVERYBODY WANTS EVERYTHING INSTANTLY! IMPROVED TECHNOLOGY JUST INCREASES EXPECTATIONS.



THESE MACHINES DON'T MAKE LIFE EASIER—THEY MAKE LIFE MORE HARASSED.



SIX MINUTES TO MICROWAVE THIS?? WHO'S GOT THAT KIND OF TIME?!



IF WE WANTED MORE LEISURE, WE'D INVENT MACHINES THAT DO THINGS *LESS* EFFICIENTLY.



© 1995 Watterston Dist. by Universal Press Syndicate

WEEK 9-17

THANK YOU